Agrisense Nexus

MVP Middleware – Data Architecture Design Documentation

Chapter 1: Purpose and Scope:

1.1: Purpose

The Agrisense Middleware serves as the business logic and data management layer between the ChirpStack LoRaWAN server and client applications. Its primary purpose is to:

- Track harvested produce as it moves through cold storage operations
- Maintain a consistent, auditable view of inventory state
- Enforce tenant isolation and operational accountability
- Expose clean, reliable APIs to frontend applications

The middleware is designed to support multi-tenant cold storage operations, where each tenant manages customers, harvest arrivals, transfers, collections, and losses independently.

1.2: Scope

The middleware is responsible for:

- Modeling business operations related to harvested produce (arrivals, transfers, collections, losses, spoilage)
- Maintaining a single source of truth inventory ledger for quantity and location changes
- Enforcing tenant-first data isolation at the database level
- Associating operational actions with users for auditability
- Interpreting sensor data produced by devices attached to cold storage units

1.3: Out of Scope

The middleware intentionally does not:

- Manage LoRaWAN device configuration, profiles, or applications
- Maintain physical sensor or device inventory lifecycles
- Perform long-term analytics or reporting (to be handled via a data warehouse)
- Compute inventory state from business tables

ChirpStack remains the system of record for device management and network operations, while the middleware focuses strictly on business correctness and inventory consistency.

Chapter 2: Core Design Principles

2.1: Tenant-First Architecture

The system is designed as a tenant-first, multi-tenant platform.

- Every piece of business data belongs to exactly one tenant
- Tenant ownership is anchored at the harvest arrival level
- All downstream entities (transfers, collections, losses, spoilage, inventory entries) derive tenant context from the arrival

Tenant isolation is enforced at the database level using Row-Level Security (RLS), with the middleware responsible for setting the tenant execution context per request.

2.2: Separation of Business Intent and Inventory State

The data model strictly separates operational intent from physical inventory changes. Business tables describe why an action occurred and who initiated it, while inventory state changes are recorded separately in a dedicated ledger.
This separation prevents data duplication, avoids conflicting sources of truth, and allows business workflows to evolve independently of inventory accounting.

2.3: Single Source of Truth for Inventory

All inventory quantities and locations are recorded exclusively in the inventory ledger.

- Inventory is represented as signed quantity deltas.
- The ledger is append-only and auditable.
- Current and historical stock state is derived by replaying ledger entries

No business table stores or computes inventory quantities.

2.4: Explicit Operational Accountability

Every business action is explicitly associated with:

- A user who initiated or handled the action
- A timestamp representing when the action occurred

This ensures traceability, supports audits, and enables clear responsibility assignment for all operational events.

2.5: Controlled Integration with External Systems

The middleware integrates with ChirpStack in a loosely coupled manner.
ChirpStack remains the authority for device and network configuration. The middleware consumes decoded data and applies business validation. Device and sensor modeling is limited to declared capabilities, not asset management.
This approach minimizes coupling while preserving flexibility.

Chapter 3: Conceptual Model Overview

The data model is organized around a small number of clearly defined concepts, with responsibilities explicitly separated to ensure consistency, auditability, and long-term maintainability.

3.1: Harvest Arrival as the Root Identity

A harvest arrival represents the atomic unit of harvested produce within the system. Each arrival corresponds to a single intake of produce.

- An arrival belongs to exactly one customer and one tenant
- All subsequent operations reference the arrival as their point of origin

The harvest arrival is the ownership and identity anchor for inventory and business events.

3.2: Table Classifications

The schema is intentionally divided into two distinct classes of tables:

Business Intent Tables
Business Intent tables capture operational actions and workflows performed by users or systems. These include:

- Harvest arrivals
- Transfers between cold storage units
- Customer collections
- Loss and spoilage events

Key characteristics:

- One row represents one business action
- May include lifecycle or status fields
- Store context, responsibility, and timing
- Do not store inventory quantities or locations

These tables answer the question:
“What was intended or performed, and why?”

State-Changing Ledger Table

The Inventory Ledger records all physical changes to inventory state.

Key characteristics:

- Append-only
- Signed quantity deltas
- Explicit storage location per entry
- Linked to the originating business event

The ledger answers the question:

"What physically changed, where, and by how much?"

3.3: Relationship Between Intent and State

Business Intent tables and the Inventory Ledger are intentionally decoupled. Their relationship is as follows:

- A single business action may produce zero, one, or multiple ledger entries
- Ledger entries never initiate business workflows
- Inventory state is never inferred from business tables

This separation guarantees that inventory calculations remain correct even as business processes evolve.

3.4: Conceptual Flow

At a high level, system behavior follows this pattern:

Business Action → Inventory Ledger Entry → Derived Inventory State

Chapter 4: Entity Relationships & Ownership

This chapter describes how ownership, tenancy, and data access are modeled and enforced across the system.

4.1: Ownership Hierarchy

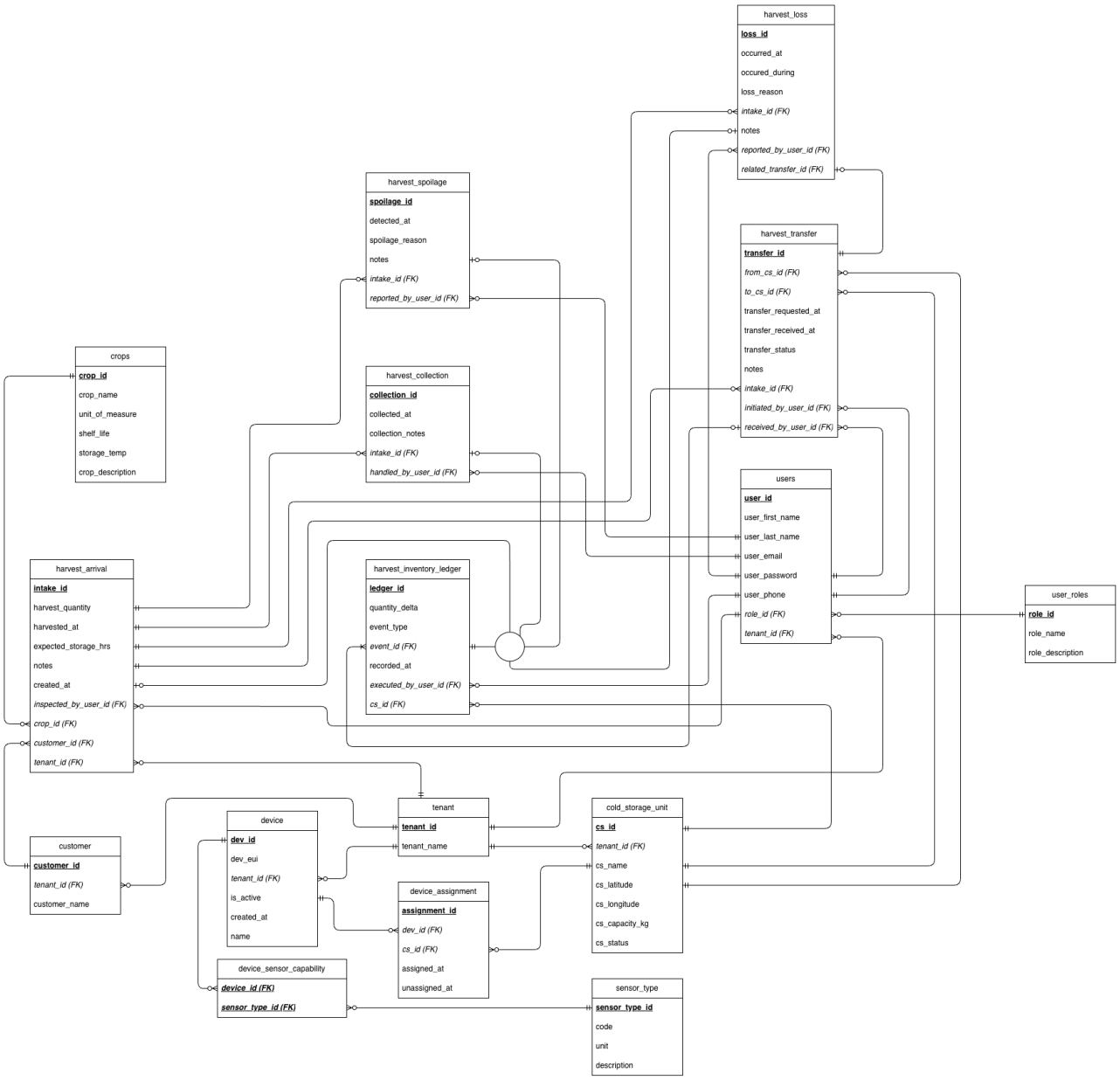
All business data in the system follows a single, consistent ownership hierarchy:

Tenant → Customer → Harvest Arrival → Business Events → Inventory Ledger

- A tenant represents an independent organization using the platform
- A customer belongs to exactly one tenant
- A harvest arrival belongs to exactly one customer and one tenant
- All downstream entities reference the harvest arrival

Ownership is defined once and derived consistently everywhere else.

Below is the ER Diagram:



4.2: Arrival-Centered Ownership Model

The harvest arrival is the root ownership entity for all operational and inventory data. Business events (transfer, collection, loss, spoilage) reference the arrival. Inventory ledger entries reference both the arrival and the originating business event. Tenant and customer context is always derived from the arrival. As a result, tenant and customer identifiers are not duplicated across downstream tables.

4.3: Referential Integrity

Ownership consistency is enforced through:

- Foreign key constraints between business tables and the harvest arrival
- Foreign key constraints between arrivals, customers, and tenants
- Database-level Row-Level Security policies

Cross-tenant data access is structurally impossible.

4.4: Querying by Tenant and Customer

Tenant- and customer-scoped queries are performed by joining through the harvest arrival.
This approach avoids redundant ownership fields, prevents desynchronization across tables and preserves a single authoritative ownership path.

4.5: Rationale for Non-Duplication

Duplicating tenant or customer identifiers across multiple tables was intentionally avoided because:

- It increases the risk of inconsistent data
- It complicates enforcement of ownership invariants
- It weakens audit guarantees

Instead, ownership is centralized and enforced through relational constraints and database policies. This ownership model forms the foundation for secure multi-tenancy, auditable operations, and consistent inventory tracking.

Chapter 5: Inventory Ledger Design

The Inventory Ledger is the authoritative record of all physical inventory changes within the system. It is designed to guarantee consistency, auditability, and correctness across all operational workflows.

5.1: Purpose of the Inventory Ledger

The Inventory Ledger exists to provide a single source of truth for:

- Inventory quantities
- Inventory locations
- Historical stock movements

All current and historical inventory states are derived exclusively from ledger entries. No other table stores or computes inventory state.

5.2: Ledger Semantics

Each row in the inventory ledger represents an atomic, irreversible change to inventory.
Key semantics:

- Quantity is stored as a signed delta
- Positive values increase inventory
- Negative values decrease inventory
- Each entry applies to exactly one cold storage unit
- Entries are timestamped and attributable to a user
- The ledger is append-only
- Ledger entries are never updated or deleted once written.

5.3: Relationship to Business Events

Business events and inventory ledger entries are related but not equivalent.
A business event may generate zero, one, or two inventory ledger entries, depending on the nature and outcome of the event.

- **Zero ledger entries -** Business events that do not directly affect physical inventory do not generate ledger entries. For example, the creation of a harvest arrival record does not modify inventory state by itself. Inventory changes occur only when produce is physically placed into or removed from a cold storage unit.
- **One ledger entry -** Business events that result in a single physical inventory change generate one ledger entry. Examples include Customer collections, Loss events, and Spoilage events.
- **Two ledger entries -** Only successful transfer events without loss generate two ledger entries. A negative quantity entry for transfer out from the source cold storage unit, and a positive quantity entry for transfer in to the destination cold storage unit

5.4: Supported Inventory Event Types

The ledger supports a constrained set of event types, including:

- ARRIVAL
- TRANSFER_OUT
- TRANSFER_IN
- COLLECTION
- LOSS
- SPOILAGE

5.5: Deriving Inventory State

Inventory state is always computed by aggregating ledger entries.
Examples:

- Remaining quantity for an arrival
- Stock per cold storage unit
- Historical inventory at a point in time

This design guarantees that inventory state cannot become inconsistent due to workflow changes, all stock movements are traceable to a responsible action and user, and historical corrections do not overwrite past data

Chapter 6: Business Event Tables

Business Event tables model operational actions and workflows performed within the system. They capture intent, accountability, and context, while delegating all inventory state changes to the Inventory Ledger.

6.1: General Characteristics

All Business Event tables share the following characteristics:

- One row represents one discrete operational action
- Each event references a harvest arrival as its identity anchor
- Events may have lifecycle states (e.g., initiated, completed, cancelled)
- Events are attributable to one or more users
- Events do not store inventory quantities or locations

Business Event tables describe what was done and why, not what physically changed.

6.2: Harvest Arrival

The harvest arrival represents the initial intake of harvested produce into the system. It establishes identity, tenant, and customer ownership. It also captures arrival metadata and inspection context, and serves as the reference point for all subsequent operations. The creation of a harvest arrival record does not modify inventory state by itself.

6.3: Harvest Transfer

A harvest transfer represents the intent to move produce between cold storage units. It is modeled as a single business event with a lifecycle (may be initiated, completed, or cancelled). Successful transfers without loss generate exactly two inventory ledger entries. Cancelled or failed transfers generate no ledger entries. The transfer table captures intent and responsibility, while the ledger captures physical movement.

6.4: Harvest Collection

A harvest collection represents the removal of produce by or for a customer. Each collection is a single business event. It generates exactly one negative inventory ledger entry
As a business event, it captures who handled the collection and when it occurred.
Collections always reduce available inventory.

6.5: Harvest Loss

A harvest loss represents the unrecoverable loss of produce due to operational incidents. Each loss is recorded as a discrete business event. It generates exactly one negative inventory ledger entry, and may optionally reference a related transfer event. Loss events provide accountability and auditability for stock discrepancies.

6.6: Harvest Spoilage

Harvest spoilage represents quality degradation, rendering produce unusable. It is recorded as a business event distinct from operational loss, and in the inventory ledger, it generates exactly one negative entry.

Chapter 7: Devices & Sensors

This chapter describes how sensing devices and measurements are modeled within the system, with a focus on simplicity and loose coupling to external systems.

7.1: Design Approach

The system intentionally avoids managing physical sensor or device inventories.

- Devices are treated as logical signal sources
- Sensors are modeled as capabilities, not as individual assets
- ChirpStack remains the system of record for device configuration and network operations

The middleware is responsible only for interpreting and validating data relevant to business operations.

7.2: Device Modeling

Devices are modeled at a logical level and scoped to a tenant.

- Each device belongs to exactly one tenant
- Devices are identified by their ChirpStack device identifiers
- Device records are used to associate incoming data with cold storage units

The system does not track device lifecycle details such as provisioning, maintenance, or decommissioning.

7.3: Sensor Capabilities

Rather than modeling individual sensors, the system defines sensor types and associates them with devices as capabilities.
Sensor types define the meaning and unit of a measurement. Devices declare which sensor types they support, which enables validation and interpretation of incoming readings

7.4 Device Assignment and Data Interpretation

Devices may be assigned to cold storage units to provide environmental data.

- Assignments are time-bound and auditable
- Incoming sensor readings are validated against declared device capabilities
- Measurements are stored independently of inventory data

Sensor data is used to inform monitoring, alerts, and analysis, but does not directly modify inventory state. Device and sensor modeling remain lightweight. The middleware is not tightly coupled to ChirpStack internals, and sensor data integration remains flexible as device profiles evolve

Chapter 8: Tenant Isolation & Security

The system is designed to enforce strict tenant isolation and prevent cross-tenant data access at all times.

8.1: Tenant Isolation Strategy

Tenant isolation is enforced using a defense-in-depth approach:

- Database-level enforcement provides hard guarantees
- Middleware-level enforcement provides contextual and role-based control

The primary mechanism for tenant isolation at the database level is Row-Level Security (RLS). Tenant context is set per database session. Access to rows is restricted based on tenant ownership. Policies are applied to all business and inventory tables. Because tenant ownership is anchored at the harvest arrival, RLS policies consistently derive tenant context through relational constraints.
The middleware, on the other hand, is responsible for:

- Authenticating users
- Determining tenant and role context
- Setting the tenant execution context for database access

- Enforcing role-based permissions beyond tenant scope
- Middleware checks complement, but do not replace, database enforcement.

This approach guarantees that:

- Users cannot query or modify data belonging to another tenant
- Cross-tenant access is structurally impossible at the database level
- Authorization failures result in safe, empty query results rather than data leaks

Chapter 9: Design Invariants

This chapter defines the non-negotiable rules that govern the system's correctness. These invariants must be preserved across all future changes.

9.1: Inventory Invariants

- Inventory quantities and locations must not be stored in business event tables
- All inventory state must be derived exclusively from the inventory ledger
- Inventory ledger entries are append-only and must never be updated or deleted
- Each ledger entry represents a single, irreversible physical change

9.2: Business Event Invariants

- Business event tables describe intent and accountability, not inventory state
- Business events may generate zero, one, or two inventory ledger entries, depending on the outcome
- Inventory changes must not be inferred solely from the existence of a business event
- Cancelled or failed business events must not affect inventory state

9.3: Ownership and Tenancy Invariants

- Every business event and ledger entry must be traceable to a single harvest arrival
- Tenant and customer ownership is defined once at the harvest arrival level
- Tenant and customer identifiers must not be duplicated across downstream tables
- Cross-tenant data access must be structurally impossible

9.4: Accountability Invariants

- Every business event must be attributable to a responsible user
- Every inventory ledger entry must reference both a business event and a user
- Timestamps must accurately reflect when actions occurred

9.5: Enforcement

These invariants are enforced through:

- Database constraints and Row-Level Security policies
- Middleware validation and authorization logic
- Code review and architectural governance

Violation of these invariants is considered a system defect.